

Advanced Programming Techniques

fbeit

FACHBEREICH ELEKTROTECHNIK
UND INFORMATIONSTECHNIK

Prof. Dr. Michael Lipp

fbeit

FACHBEREICH ELEKTROTECHNIK
UND INFORMATIONSTECHNIK

C++ Libraries

Purpose

- **Libraries provide re-usable functions and types**
 - No need to invent own version
 - Ensure portability
 - Well tested
- **Items from a library are made available**
 - by including a header file
 - C: `#include <someHeaderFile.h>`
 - C++: `#include <someHeaderFile>`
 - adding pre-compiled code
 - Provides implementation of functions
 - Not need for the “C++ Standard Library” (that’s what makes it “standard”)

Examples from the C++ Standard Library

- **“Function” library “math”**

- `#include <math.h> // C-functions (no need for “using namespace std”)`
- `#include <cmath> // Preferred!`
- Provides lots of functions such as `sin`, `cos` etc.
- Complete list: <http://www.cplusplus.com/reference/cmath/>

- **“Basic Type” library types “cstdint”**

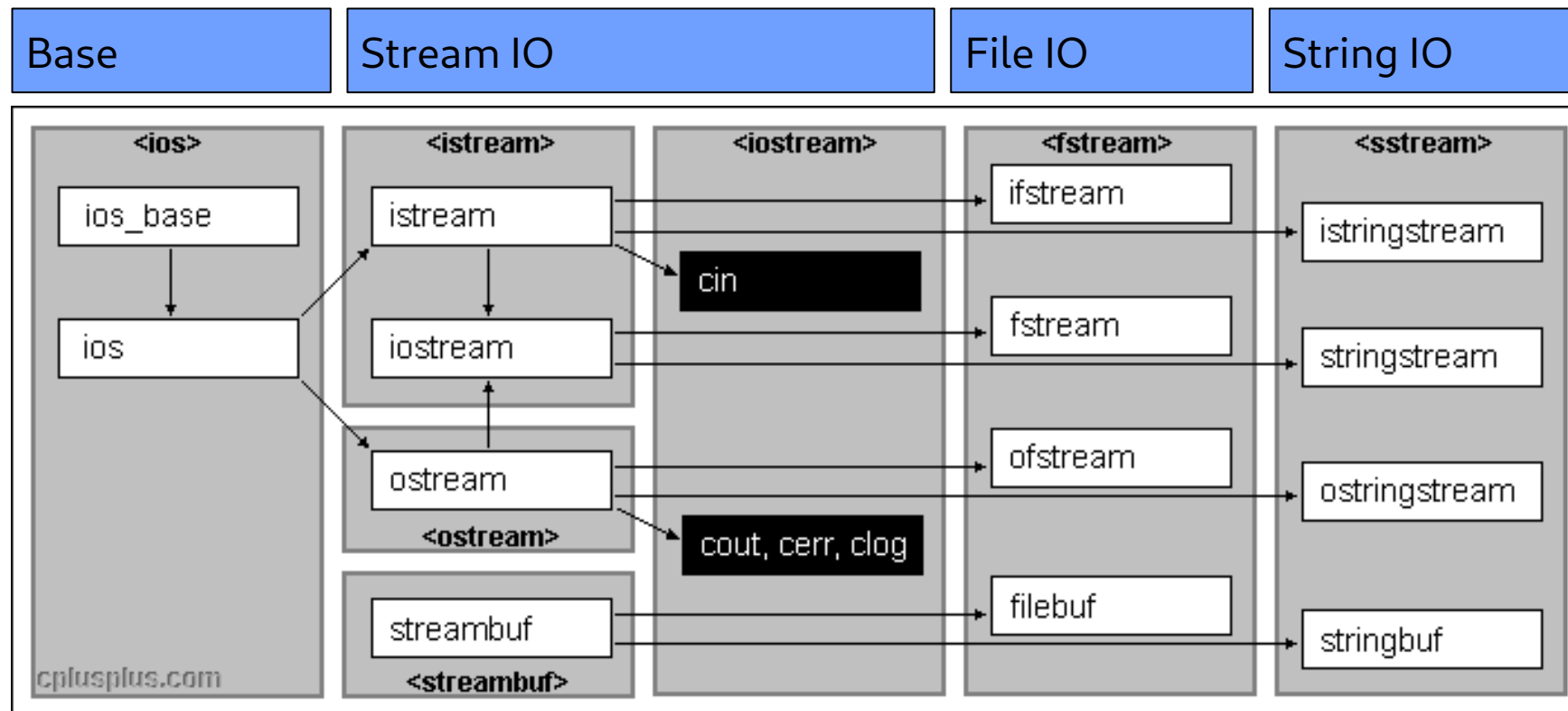
- Provides integral types with specific widths
- Examples: `int8_t`, `uint16_t`, `int32_t` etc.
- Overcomes portability problems (“`int`” has no standardized size/range)

Library class for handling strings

- Library class “string”
 - More functions than you think!
 - Have a look: <http://www.cplusplus.com/reference/string/>

Library classes for I/O

- Library class “iostream”



iostream details

- **Well known**
 - `cout`: global instance of `ostream` that streams data to console
 - `cin`: global instance of `istream` that streams data from console
- **All streams are used in the same way**
- **More types of streams**
 - `fstream` (`ofstream`, `ifstream`)
 - Write to/read from a file

Library classes for smart pointers

- The “C++ Standard Library” provides “smart pointers”
 - Problem with “raw pointers”: responsibility for objects on the heap is unknown
 - When a method returns a raw pointer to something,
 - the item pointed to may be on the stack → no action by caller required
 - the item pointed to may be on the heap and was allocated before the method call → no action by caller required
 - the item pointed has been allocated to return a result → it's the caller's responsibility to delete the object when he needs it no longer!
 - To cover all use cases, the library provides unique pointers and shared pointers

Smart pointers: `unique_ptr` (1)

- Usage

- `auto myX = unique_ptr<X>(new X());`
- Takes ownership of the object pointed to
- When variable `myX` goes out of scope (i.e., when its destructor is called) the `unique_ptr` automatically calls `delete` for the pointer passes to the constructor
- Used just like a pointer (`operator*` and `operator->` are overloaded)
 - `myX->doSomething();` // Invoke X's method
 - `X anotherX = *myX;` // Access X
- Assignment and copy constructor move ownership, therefore move-“versions” must be used (see next slide)

Smart pointers: `unique_ptr` (2)

- **Transferring ownership**

- Creating a new new `unique_ptr` as copy of an existing `unique_ptr` or assignment to a `unique_ptr` transfers the ownership
 - This implies that the “source” object of the operation is modified (no longer owns the pointer) which is normally not allowed for these operations (parameter lists are typically “...(const type& source)”)
- However, the **move** constructor and **move** assignment operator do allow modification of the source object – but are usually only invoked for temporary objects
- Solution: make the source object “look like” a temporary object:

```
unique_ptr<X> a(new X());  
unique_ptr<X> b = std::move(a);
```

Smart pointers: shared_ptr

- Usage

- Like `unique_ptr`, but `delete` is called for pointer when last `shared_ptr` is deleted
- Can be used with “rule of zero” only iff object pointed to is immutable (that’s why immutable classes are nice to have)

shared_ptr vs. unique_ptr vs. raw pointer

- Use **unique_ptr** or **shared_ptr** to indicate ownership (composition relationship)
 - Use `unique_ptr` if you are the only owner
 - Use `shared_ptr` if the “whole” in the relationship has a cardinality greater 1 (example: a wall is part of a room but can belong to two rooms)
 - Use `shared_ptr` for objects without composition relationship to “a whole”
- Use **raw pointers** to pass references to otherwise owned objects
 - Be sure that the otherwise owned object outlives the usage of the pointer
 - Prefer references

Library classes for containers

- The “C++ Standard Library” provides all typically needed container types
 - “SimpleVector” from the lecture is a typical example of a “container”
- “**<vector>**”
 - Sequence container representing arrays that can change in size
 - Contained type specified in angle brackets (`vector` is a “template class”, i.e. contained type is “left open” in definition and specified when instantiated)
 - `vector<double> myVector`
 - `vector<Fraction> myOtherVector`

<vector>

- **Sample operations**

- `push_back`: add element at end (copied or moved)
- `pop_back`: remove last element
- `insert`: add element at given position
- `erase`: remove element at given position
- `at` / `operator[]`: return reference to element at given position
- `size`: returns number of elements

Elements must be at least copy constructable and copy assignable

Specified by `iterator` (details follow)

- See <http://www.cplusplus.com/reference/vector/vector/>

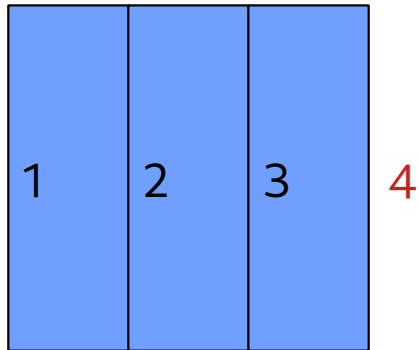
<list>

- Sequence container with constant insert time (among other properties)
 - Example: `list<int> myList;`
- Sample operations
 - Same as vector, plus
 - `push_front`: add element at beginning
 - `pop_front`: remove element at beginning
 - `sort`: sort elements (element type must override `operator<`)
- See <http://www.cplusplus.com/reference/list/list/>

Sequence container implementations

- **Vector: array based**

- Easy random access: $\text{start_address} + n * \text{sizeof}(\text{element})$
- Costly prepend/insert (requires moving many elements)
- Simple append, unless array needs to grow

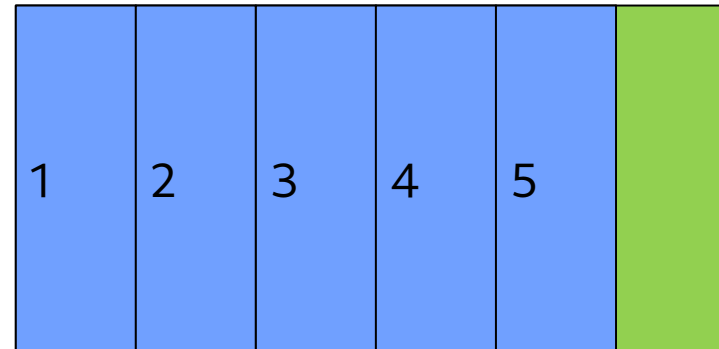


Sequence container implementations

- **Vector: array based**

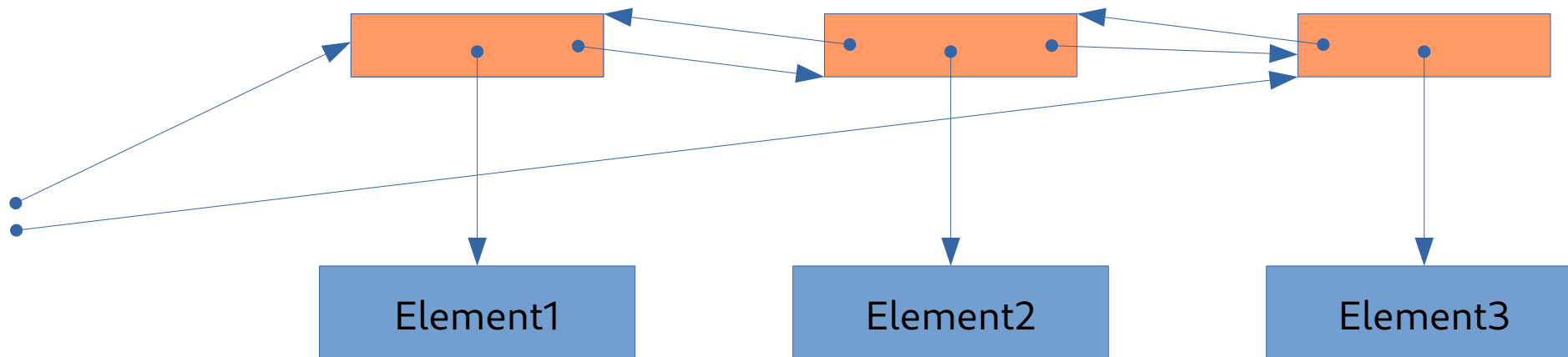
- Easy random access: $\text{start_address} + n * \text{sizeof}(\text{element})$
- Costly prepend/insert (requires moving many elements)
- Simple append, unless array needs to grow

Elements in a vector can move in memory, never store their addresses in pointers!



Sequence container implementations

- **(Doubly-linked) List: Helper structure for maintaining sequence**
 - Constant insert/remove times
 - Easily modifiable (add/remove) start/end
 - Costly random access (must follow links from start/end)



Iterators

- Very often, list elements are accessed sequentially, but using position (index) makes access “look random”

```
vector<double> mySequence = { 1, 2, 3, 4, 5, 6, 7 };  
for (unsigned i = 0; i < mySequence.size(); i++) {  
    cout << mySequence[i] << endl;  
}
```

Unsupported for list,
too expensive

- Solution: Helper class “**iterator**”
 - Efficiently marks a position, “knows” about the details of the container implementation, available for all container types
 - Can be moved forward (and sometimes backward)
 - Can be used to access elements, as marker for insert/remove...

Iterators

- Sequential access

```
list<double> mySequence = { 1, 2, 3, 4, 5, 6, 7 };  
for (list<double>::iterator iter = mySequence.begin();  
    iter != mySequence.end(); iter++) {  
    cout << *iter << endl;  
}
```

Returns an iterator
positioned at the
first element

Returns an iterator
positioned **after** the
last element

Iterators use operator
overloading to make
them “look like pointers”

Iterators

- Sequential access simplified

```
list<double> mySequence = { 1, 2, 3, 4, 5, 6, 7 };  
for (auto iter = mySequence.begin();  
     iter != mySequence.end(); iter++) {  
    cout << *iter << endl;  
}
```

- “**auto**” used as type

- Instructs the compiler to infer type from initialization expression
- Use with care! The compiler might know the type, but does the reader?

Iterators

- Iterators are usually used for iterating over a container
- Sometimes, library functions use iterators for appending or inserting elements into a container
 - Such iterators can be created from container
 - `std::back_inserter(container)` creates a `back_inserter_iterator`, assigning to the iterator calls `push_back` on the container
 - `std::front_inserter(container)` creates a `front_inserter_iterator`, assigning to the iterator calls `push_front` on the container
 - `std::inserter(container)` creates an `inserter_iterator` from a container and a “normal” iterator, assigning to it inserts at the position denoted by the “normal” iterator

const Iterators

- How about const containers?

Special for-loop

- Sequential access simplified even further

```
list<double> mySequence = { 1, 2, 3, 4, 5, 6, 7 };  
for (double value: mySequence) {  
    cout << value << endl;  
}
```

- **value** assumes the value of each element in the sequence, one after the other
 - The variable can also be declared as reference type to allow modifications of elements in sequence
 - This kind of for loop can also be used for arrays

More functions

- Some functions that you would expect to be member functions are defined based on iterators (i.e. for all container types) in **<algorithm>**

- Finding elements in containers (requires overloaded operator==)

```
list<double> mySequence = { 1, 2, 3, 4, 5, 6, 7 };  
auto position = find(mySequence.begin(), mySequence.end(), 5);  
cout << *position << endl;
```

- Replace elements in containers (requires overloaded operator==)

```
list<double> mySequence = { 1, 2, 3, 4, 5, 6, 7 };  
replace(mySequence.begin(), mySequence.end(), 5, 10);  
for (auto value: mySequence) {  
    cout << value << endl;  
}
```

- ...